AXK Network

October 2026

TECHNICAL REPORT

Verification & Settlement Layer on Stellar

Four Soroban smart contracts and a Frappe v15 application for verified cross-border agricultural trade between Rwanda, the DRC and Uganda.

STUDENT
Oghenetejiri Jesse

SUPERVISOR
Junior Turatsinze

PROGRAMME
BSE Software Engineering Capstone

INSTITUTION
African Leadership University

LIVE SYSTEM
capstone.axk.org

DEMO VIDEO
youtu.be/MrSXiJG_MAU

REPOSITORY
github.com/tejirijesse/axk-open-source-capstone

OFFLINE SIMULATION
axk-open-source-capstone.vercel.app

CONTENTS

What this report covers

01 Problem and approach 1

02 Scope: planned versus delivered 1

03 Functionality and architecture 2

3.1 Controls on the application layer 2
3.2 Two ways to run it 2
3.3 What was exercised on testnet 2
3.4 Standards and interoperability 2

04 Testing 3

4.1 Failure-mode tests (money and integrity paths) 3
4.2 What testing does not cover 3

05 Performance and browser compatibility 4

5.1 Payload and response time (measured) 4
5.2 Browser compatibility 4
5.3 Deployment and platform 4

06 Code quality and object-oriented design 5

6.1 Contracts: types, encapsulation, typed errors 5
6.2 Application: model, service layer, boundary 5
6.3 Algorithms worth reading 5
6.4 Project hygiene 5

01 Problem and approach

Cross-border agricultural trade between Rwanda, the DRC and Uganda fails at two points: a buyer cannot verify a lot's quality, custody and container integrity along the corridor, and settlement is slow and hard to trust. This project puts both in one workflow. A cooperative registers a lot, an independent inspector signs it off on-chain, a buyer locks payment in escrow, a carrier records dispatch under a container seal, the port checks the seal and records clearance, and on delivery confirmation the contracts release payment, split it 40/40/20 between members and write an attestation anyone can check through a QR code. Four Soroban contracts (lot registry, escrow, splitter, attestation) hold the rules; a Frappe v15 app holds the roles, screens and signing.

02 Scope: planned versus delivered

Scope item	Status	Evidence in the repository	Limit or change from plan
1. Lot registry	Delivered	`contracts/lot_registry`, 18 tests; lot id is `sha256(cooperative || lot_ref)`, so an id cannot be squatted; registration is write-once.	Only a digest of the lot record goes on-chain.
2. Independent inspection	Delivered	`/trade/inspect` queue; inspector must be on the registry's allow-list; test `a_cooperative_cannot_inspect_its_own_lot`. Offline capture in ODK (`odk/`, `odk.py`).	ODK import was added beyond the first plan, for sites without signal.
3. Escrow	Delivered	`contracts/escrow`, 27 tests; asks the registry whether the lot passed before any transfer; `create` has no payee argument, `release` takes only a trade id.	Rebuilt around the registry (inspection gate, one live escrow per lot).
4. Logistics and seal	Delivered	`/trade/logistics`; `chain.dispatch_logistics` attests the waybill digest.	Attestation is signed by the AXK verifier key for the carrier. Carrier self-signing is roadmap LOG-4.
5. Port gate-in	Delivered	`/trade/port`; `chain.receive_port` compares the arriving seal to the waybill and attests the port receipt.	The seal match is an application check, not a contract rule (roadmap PORT-1).
6. Settlement and split	Delivered	`contracts/splitter`, 12 tests; `chain.settle` releases escrow, records the split and writes the final attestation.	Token is a Stellar test asset standing in for RWF.
7. Public trace and explorer	Delivered	`/trace` (no login, no personal data, QR, GS1 EPCIS 2.0 download via `epcis.py`); `/chain` live Soroban event reader (`stellar_live.py`).	The five-year history on `/chain` is labelled synthetic seed data.

Not done: the field pilot in Kigali has not been run, so nothing here is pilot evidence. Mainnet use needs an external contract audit first. Both are stated in the README and `docs/security.md`.

03 Functionality and architecture

Application (Frappe v15)

MariaDB 10.11, Redis
DocType `AXK Chain Trade`
Role endpoints in `trade_api.py`
Pages `www/trade/*`, `/trace`, `/chain`

Custodial signer (`chain.py`)

One testnet identity per role
Calls `stellar-cli` with argument lists, no shell
Canonical JSON then SHA-256
Contract error codes mapped to messages

Stellar testnet (Soroban)

Lot registry, escrow, splitter, attestation
Rust, `soroban-sdk` 22, `wasm32v1-none`
RWF test token (Stellar Asset Contract)

Each of the six trade steps is one HTTP call from a role screen to a whitelisted function in `trade_api.py`. The function checks the login and role, calls the matching function in `chain.py`, which builds the canonical record, hashes it, invokes the contract and stores the transaction hash on the `AXK Chain Trade` document. If the chain call fails, the step fails and nothing is recorded, so the database never claims something the ledger did not accept. Reads for the public pages come from the stored hashes, re-checked against the contracts by `matches_on_chain`, and the browser re-hashes the records itself.

3.1 Controls on the application layer

- **Access guard:** `_guard()` in `trade_api.py` allows a signed-in user with the AXK Capstone Demo role (or System Manager). Demo mode (`axk_demo_mode`) only works for requests made directly on loopback; anything through a proxy is refused. The hosted site runs with demo mode off.
- **Throttle:** `_throttle()` caps 30 signed transactions per user per hour, keyed by user and not by IP, so a shared reviewer login cannot spam testnet.
- **Keys:** per-role Stellar identities live on the server under the service user (directory 700, files 600, outside git). The browser never holds a key. See `docs/handover.md` and `docs/security.md`.
- **Privacy:** `verify.py` exposes a fixed list of PII-free fields to the public trace page.

3.2 Two ways to run it

Full stack (`make run-frappe`): every action is a real Stellar testnet transaction; this is what capstone.axk.org and the demo video show. **Offline simulation** (`make run`, `scripts/server.py`, Python only): the same screens with locally generated hashes, labelled "Offline simulation, not on-chain" on every page and never linked to a block explorer. It exists so the screens can be tried and the end-to-end tests run without a bench.

3.3 What was exercised on testnet

`docs/deployments.md` records a run of `scripts/demo.sh` on 29 September 2026 with the contract IDs, transaction links and the exact errors the contracts returned: funding before inspection gives `Error(Contract, #10) LotNotApproved`; a cooperative inspecting its own lot gives `#3 NotAnInspector`; a second escrow on a sold lot gives `#11 LotInEscrow`; a stranger calling release is refused.

3.4 Standards and interoperability

Every lot's public history exports as a GS1 EPCIS 2.0 document (`epcis.py`, validated against the official schema when `jsonschema` is installed). Inspections captured offline in ODK are imported through `odk.py` and signed on import.

04 Testing

Four suites cover three approaches: unit tests of contract logic in a mocked Soroban environment, end-to-end and failure-mode tests of the application, and standards/offline-capture tests. Counts below are from running the suites on 30 October 2026.

Suite	Tests	What it checks	Run and result
Lot registry `contracts/lot_registry`	18	Write-once registration, allow-listed inspectors, cooperative cannot inspect its own lot, unknown lot returns an error instead of panicking.	`make test-contracts` (`cargo test`) **65 passed, 0 failed**
Escrow `contracts/escrow`	27	Inspection gate, fixed payee, verifier-only release, deadline refund, one live escrow per lot, zero and negative amounts.	
Splitter `contracts/splitter`	12	Shares sum to 100%, configuration is fixed once set, remainder goes to the last share so no unit is lost.	
Attestation `contracts/attestation`	8	Digest is write-once, only the attester writes, unknown trade is an error.	
End to end `scripts/test_trade.py`	7	Overview and personas, full lifecycle, inspection rejection, seal tamper detection, invalid state transitions, unknown lot, web routes and trace rendering.	`make test-python` against the offline simulation: **7 passed**
EPCIS export `scripts/test_epcis.py`	8	Event structure and schema validation of the GS1 EPCIS 2.0 document.	`make test-epcis`: **8 passed**
ODK offline `scripts/test_odk.py`	9	Form, importer and ODK Central sync; imports new submissions once and skips rejected ones.	`make test-odk`: **8 passed, 1 skipped** (live signing needs `AXK_ODK_LIVE=1`)

Total: 89 tests defined; 88 run, 88 passed, 1 skipped by design. The 14 contract guarantees in `docs/security.md` each name the test that pins them.

4.1 Failure-mode tests (money and integrity paths)

Because escrow and settlement move value, failure paths were written before happy-path polish. Examples from the contract suite, in `contracts/*/src/test.rs`:

```
#[test]
fn a_cooperative_cannot_inspect_its_own_lot() { /* inspect by the owner returns Error::NotAnInspector */ }
#[test]
fn inspecting_an_unknown_lot_is_an_error_not_a_panic() { /* typed error, no abort */ }
#[test]
fn an_unknown_trade_id_is_an_error_not_a_panic() { /* escrow and splitter */ }
```

The end-to-end suite adds the application side: a rejected lot cannot be funded, a container arriving with a different seal than the waybill is flagged and blocked, a buyer cannot confirm delivery before dispatch, and a lot that was never registered returns an error with no state change. On Stellar testnet the same refusals were observed as real contract errors (section 3.3).

4.2 What testing does not cover

The end-to-end tests ran here against the offline simulation; against the full stack each step is a testnet transaction and takes seconds, so that run is done by hand (`make demo`, the demo video). There is no load test, no browser-automation suite and no external audit of the contracts.

05 Performance and browser compatibility

5.1 Payload and response time (measured)

Item	Raw	gzip
`public/js/trade.js` (role screens)	63.8 KB	16.2 KB
`public/js/chain.js` (explorer, hash verifier)	28.3 KB	8.7 KB
`public/css/axk.css` + `chain.css`	29.2 KB	7.1 KB

The screens use no frontend framework and no chart library (SVG is hand-built), so the heaviest page ships about 32 KB of gzipped assets. Against the offline simulation on a laptop, `/trade`, `/trace` and `/chain` each returned in about 1 ms (2.4 KB, 5.3 KB and 6.6 KB gzipped HTML). Records are re-hashed in the browser with the WebCrypto SHA-256 API. These are server-side local figures, not field measurements over 3G; no mobile-network test was run.

On the full stack the time is dominated by Stellar: each action is a signed transaction submitted through `stellar-cli` and confirmed in a ledger close, which takes a few seconds on testnet (not benchmarked here). The signer therefore runs a call with a 90 second timeout and reports a clear "Stellar testnet did not answer in time" error instead of hanging. Wallet balances are cached (`forget_balances` clears them after a payment) so role screens open without a chain read per row. The chain explorer reads events from Soroban RPC, which only keeps about 7 days of history; the pilot plan adds an indexer for that.

5.2 Browser compatibility

The front end is plain HTML, CSS and JavaScript with `fetch`, inline SVG and WebCrypto, all of which are available in current Chrome, Edge, Firefox and Safari, and in mobile Chrome and Safari. The responsive layout targets phones used by cooperative and field staff. No formal cross-browser test matrix was run, and I do not claim one. One known quirk is recorded in the README: Safari does not resolve `*.localhost`, so local use needs `http://localhost:8000`.

5.3 Deployment and platform

Tested on macOS 15 (Apple silicon) with Homebrew, and on the hosted Ubuntu 24.04 VM (nginx, supervisor, Frappe v15, MariaDB 10.11, Redis) at capstone.axk.org. Tool versions: Rust stable with target `wasm32v1-none`, `soroban-sdk` 22, `stellar-cli` 28, Python 3.11. The hosted site has demo mode off and requires a login to sign. Operation, restart and checks are in `docs/handover.md`.

06 Code quality and object-oriented design

The code base is about 2,300 lines across the four contracts and the Frappe app modules, with no module doing two jobs. The object-oriented structure sits in three places: the contracts, the Frappe document model, and the layering between them.

6.1 Contracts: types, encapsulation, typed errors

Contract and file	Types	Design point
`contracts/lot_registry/src/lib.rs`	`LotRegistry` (contract), `Lot`, `Inspection` (structs), `LotStatus` (enum), `Error`	State is private to the contract; the only way to change a `Lot` is a method that checks the caller. Status moves forward only.
`contracts/escrow/src/lib.rs`	`Escrow` contract, trade record, `Error` with 10 named codes (`LotNotApproved = 10`, `LotInEscrow = 11`)	Depends on the registry only through its client interface (cross-contract call), so the two evolve separately.
`contracts/splitter/src/lib.rs`	`Splitter` contract, `Share` struct (party, basis points), `Error`	Shares are fixed per trade at configuration time and cannot be edited afterwards.
`contracts/attestation/src/lib.rs`	`Registry` contract, `Attestation` struct, `Error`	Write-once digest per trade; only the attester can write.

Failures are values, not panics: each contract defines a `#[contracterror]` enum, and `chain.py` maps the numeric code back to a readable message (`CONTRACT_ERRORS`), so an operator sees "lot not approved" rather than "Error(Contract, #10)".

6.2 Application: model, service layer, boundary

- **Model:** `AXKChainTrade(Document)` in `frappe_app/axk_capstone/axk_capstone/axk_capstone/doctype/axk_chain_trade/axk_chain_trade.py` is the persistent record of one trade (79 fields: lot record and digest, report digest, one transaction hash per on-chain step). Permissions and field rules are declared in its JSON schema.
- **Service layer:** `chain.py` exposes one function per trade step (`register_lot`, `inspect_lot`, `fund_escrow`, `dispatch_logistics`, `receive_port`, `settle`) that takes the document and updates it. All process execution is behind a single private function `_run`, so the transport (`stellar-cli` today) is replaceable in one place. `ChainError` is the single exception type crossing the boundary; `_fail` in `trade_api.py` records it on the document.
- **Boundary:** `trade_api.py` holds only whitelisted endpoints plus `_guard` and `_throttle`; it contains no contract details. `verify.py`, `epcis.py` and `odk.py` each own one concern (public lookup, standards export, offline import).

I am not claiming a Strategy or Repository class hierarchy: the Python side is a thin service module around one Document class and one exception class, which is the idiomatic Frappe shape. The inheritance and encapsulation come from the Frappe `Document` base and from the Soroban contract types.

6.3 Algorithms worth reading

Canonical digest (`chain.py`)

```
def canonical(obj):
    return json.dumps(obj, sort_keys=True,
        separators=(",", ":"), ensure_ascii=False)
def sha256_hex(text):
    return hashlib.sha256(text.encode("utf-8")).hexdigest()
```

Sorted keys and no whitespace make the same record hash the same in Python, in the browser re-check and in the EPCIS export.

Lossless split (`splitter`)

```
let part = if i as u32 == last {
    amount - paid          // remainder
} else {
    amount * share.bps / BPS
};
paid += part;
```

Integer division truncates; the last share receives what is left, so the parts always add up to the amount paid in.

6.4 Project hygiene

MIT licence, `CONTRIBUTING.md`, `ROADMAP.md` with starter-to-advanced tracks per trade stage, `SECURITY.md`, `CHANGELOG.md`, issue and PR templates, and a `Makefile` with one command per task. Secrets stay out of git: testnet identities are in the stellar-cli keystore on the server. Deliverables: this report, the repository, the demo video and the hosted system.